

N-Gram Language Models and Online Prediction

Diego Besada Rodríguez

[\[My code\]](#) [\[Ngram files\]](#) [\[Pierre Nugues' Template\]](#) [\[Norvig Notebook\]](#)

September 2026

1 Objectives

An n -gram language model assigns a probability to a sequence of words by assuming that each word depends only on the previous $n - 1$ words. These models address tasks such as next-word prediction, spell checking, and others. The objective of this laboratory is to build **unigram**, **bigram** and **trigram** models over the corpus of *Selma Lagerlöf*, to expose the resulting probabilities and derived measures (entropy rate and perplexity), and to use them to predict the next word given a partially typed text.

2 Method and Program Structure

2.1 Corpus Preparation

The corpus is downloaded and loaded as a single string. A first tokenizer extracts word tokens with the Unicode regular expression `\p{L}+`, which matches runs of letters and discards punctuation, digits and whitespace, yielding **923,485** tokens and **44,256** unique tokens (**41,032** lowercased).

Two normalizations are then applied before counting. The function `clean` keeps only letters and sentence punctuation (`. ; : ? !`). The function `segment_sentences` marks sentence boundaries by inserting `</s>` and `<s>` between a punctuation mark and the following capital letter, wraps the text in `<s>...</s>`, removes the punctuation and lowercases everything. The corpus contains **59,047** sentences.

2.2 Counting the n -grams

Unigrams, bigrams and trigrams are counted into dictionaries whose keys are the n -grams (plain strings for unigrams, tuples for bigrams and trigrams): $C(w_i)$, $C(w_{i-1}, w_i)$ and $C(w_{i-2}, w_{i-1}, w_i)$. Each dictionary is exported to a JSONL file, one object per line, and Table 1 summarizes the collections.

File	Unique n -grams	Size
<code>unigrams.jsonl</code>	41,031	1.5 MB
<code>bigrams.jsonl</code>	320,122	13.9 MB
<code>trigrams.jsonl</code>	671,263	34.0 MB

Table 1: Vocabulary size and file size of each exported n -gram collection.

2.3 Language Models

Unigram model. A sentence is modeled as a sequence of independent words, so its probability is the product of the individual word probabilities, estimated by their relative frequency:

$$P(w_1 \dots w_n) = \prod_{i=1}^n P(w_i), \quad P(w_i) = \frac{C(w_i)}{N}, \quad (1)$$

where N is the total number of tokens in the segmented corpus.

Bigram model. Each word is conditioned on the previous one:

$$P(w_1 \dots w_n) = \prod_{i=1}^n P(w_i \mid w_{i-1}), \quad P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}. \quad (2)$$

When a bigram is unseen ($C(w_{i-1}, w_i) = 0$), we **back off** to the unigram probability:

$$P(w_i \mid w_{i-1}) = \frac{C(w_i)}{N} \quad \text{if } C(w_{i-1}, w_i) = 0. \quad (3)$$

The start marker `<s>` is prepended only so that the first real word can be conditioned on it, i.e. the first factor is $P(w_1 \mid \text{<s>})$. Since `<s>` is never predicted, there is no factor $P(\text{<s>})$; the product therefore contains one factor per real word, which after prepending is $n = \text{len}(\text{sent_words}) - 1$.

Derived measures. To compare sentences of different lengths we report the geometric mean, the entropy rate and the perplexity:

$$\text{GM} = P^{1/n}, \quad H = -\frac{\log_2 P}{n}, \quad \text{PP} = 2^H. \quad (4)$$

2.4 Next-Word Prediction

Given a prefix, the predictor selects the most frequent bigram or trigram whose context matches the text already typed and ranks the candidates by their count. For a partial word, it additionally filters candidates with `startswith`.

3 Results and Conclusion

The models are evaluated on the sentence `det var en gång en katt som hette nils </s>`. The bigram model lowers the perplexity from **423.68** to **72.85** with respect to the unigram model, which confirms that conditioning on the previous word captures useful structure.

Model	$P(\text{sentence})$	Geometric mean	Entropy rate	Perplexity
Unigram	5.365×10^{-27}	0.002360	8.7268	423.68
Bigram	2.376×10^{-19}	0.013727	6.1868	72.85

Table 2: Sentence probability and derived measures for the unigram and bigram models.

Table 3 lists the probabilities used to score the sentence under each model.

Unigram		Bigram	
Word	$P(w_i)$	Bigram	$P(w_i w_{i-1})$
det	0.020266	(<s>, det)	0.096059
var	0.011608	(det, var)	0.181874
en	0.012975	(var, en)	0.058892
gång	0.001279	(en, gång)	0.052242
en	0.012975	(gång, en)	0.015015
katt	0.000015	(en, katt)	0.000444
som	0.015638	(katt, som)	0.125000
hette	0.000093	(som, hette)	0.002763
nils	0.000084	(hette, nils)	0.000084
</s>	0.056691	(nils, </s>)	0.022989

Table 3: Per-token probabilities for the example sentence under the unigram (left) and bigram (right) models.

4 Word Segmentation with N-Grams

N-gram models are also the basis of word segmentation, the task of recovering word boundaries from text written without spaces. Using the unigram (`segment`) and bigram (`segment2`) functions from Norvig’s notebook, we segmented the following strings as follows:

Input	Unigram	Bigram
ihavealotofworktodotoday	alot	a lot
shelookedatmeforallongtime	along	a long
atleastwetriedourbest	atleast	at least
canwetalkaboutitanotherday	talkabout	talk about

Table 4: Boundary recovered by the bigram model where the unigram model merged two words.

The unigram model maximizes $\prod_i P_1(w_i)$ and ignores word order, so it prefers whichever single token is the most frequent and merges two words whenever their concatenation is common. The bigram model maximizes $\prod_i P(w_i | w_{i-1})$; conditioning on the previous word, it recovers the correct boundary. It is not always better (`smallandinsignificant` becomes `in significant`), and an out-of-vocabulary word still receives probability zero.